

BÀI TẬP LỚN · HỌC MÁY CƠ BẢN

Dự đoán giá kim cương

Xây dựng, tinh chỉnh và so sánh 4 mô hình hồi quy để tự động định giá kim cương, đóng gói theo quy trình MLOps end-to-end.

Nguyễn Huy Hoàng · Hoàng Thu Hương — Mã lớp 12523W.2

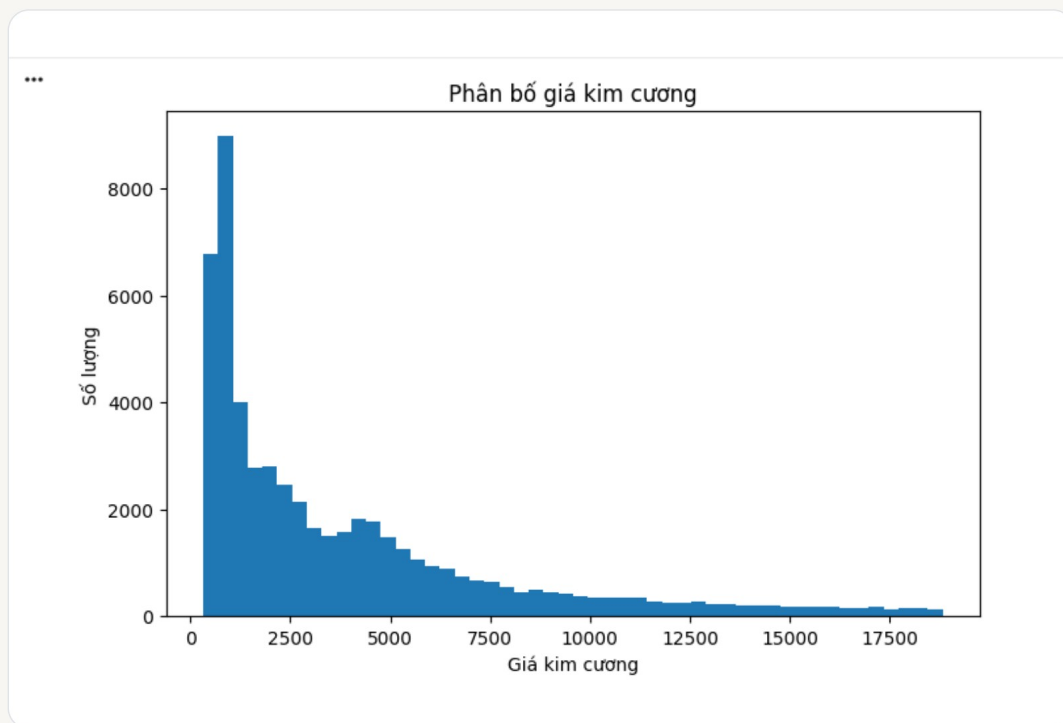
GVHD: ThS. Nguyễn Đức Tuấn Anh · Trường Đại học Sư phạm Kỹ thuật Hưng Yên

Định giá kim cương bằng học máy: từ dữ liệu đến sản phẩm

Giá kim cương phụ thuộc phi tuyến vào nhiều đặc trưng (carat, cut, color, clarity...). Đề tài xây dựng một quy trình khép kín, thay cho cách định giá cảm tính truyền thống.

- 1. Dữ liệu.** 53.940 viên kim cương (Diamonds dataset), 9 đặc trưng carat, cut, color, clarity, depth, table, x, y, z → mục tiêu price.
- 2. EDA (5 biểu đồ).** Phân bố giá lệch phải, carat tương quan mạnh với giá, cut/color/clarity ảnh hưởng phi tuyến và không đơn điệu.
- 3. Tiền xử lý.** Loại bản ghi lỗi kích thước 0 → chia train/test 80/20 → One-Hot cho biến hạng mục, gói trong Pipeline chống rò rỉ dữ liệu.
- 4. Huấn luyện.** 4 mô hình khác họ thuật toán, tinh chỉnh bằng GridSearchCV (CV = 3, tối ưu RMSE).
- 5. Đánh giá & triển khai.** So sánh R^2 , MAE, RMSE trên tập test → chọn mô hình tốt nhất → đóng gói joblib + FastAPI + Docker.

Phân bố giá kim cương



Hình cho thấy gì?

Phân phối giá lệch phải rõ rệt: phần lớn kim cương có giá dưới 5.000 USD, đuôi phải kéo dài tới gần 19.000 USD.

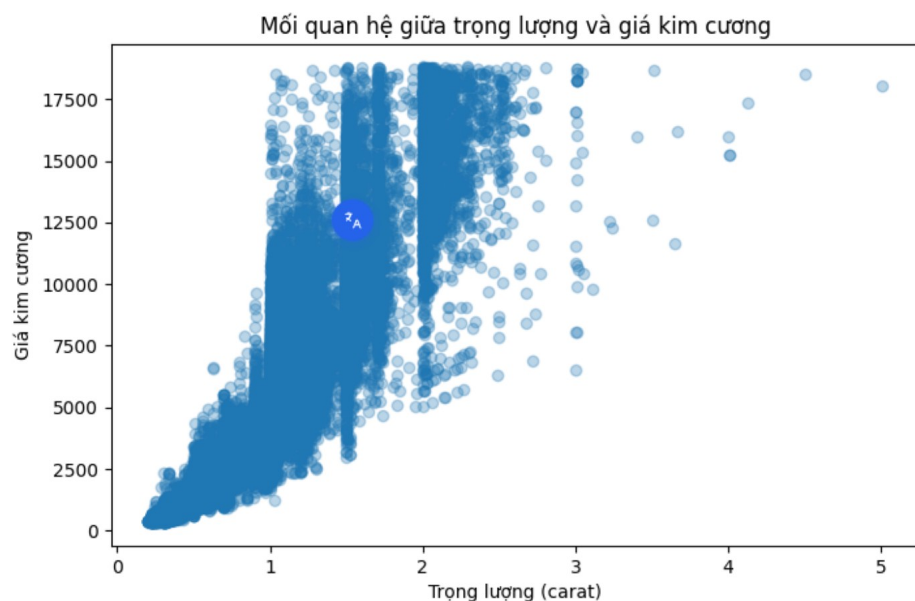
Ý nghĩa với bài toán?

Đa số kim cương phổ thông có giá vừa phải; số ít viên trọng lượng lớn hoặc chất lượng cao có giá rất cao → dữ liệu mất cân đối.

Quyết định xử lý

Ưu tiên các mô hình cây / ensemble (ít nhạy với phân phối lệch) thay vì chỉ dùng hồi quy tuyến tính thuần.

Mối quan hệ giữa trọng lượng (carat) và giá



Hình cho thấy gì?

Quan hệ đồng biến mạnh nhưng phi tuyến: giá tăng nhanh theo carat, có dạng "bậc thang" quanh các mốc carat tròn (1.0, 1.5, 2.0...).

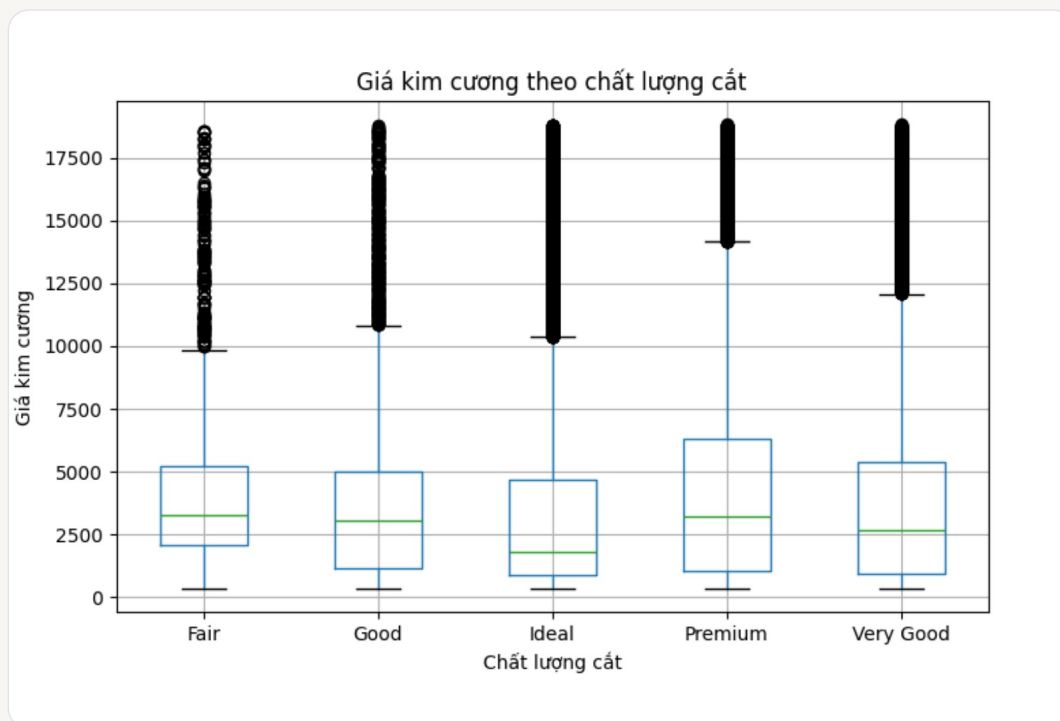
Ý nghĩa với bài toán?

Carat là đặc trưng ảnh hưởng lớn nhất, nhưng cùng một mức carat vẫn có độ phân tán giá lớn do cut/color/clarity chi phối thêm.

Quyết định xử lý

Giữ carat làm đặc trưng số quan trọng nhất; chọn mô hình phi tuyến (cây, rừng ngẫu nhiên, gradient boosting) để nắm bắt quan hệ dạng bậc thang này.

Giá kim cương theo chất lượng cắt (Cut)



Hình cho thấy gì?

Giá trung vị KHÔNG tăng đơn điệu theo Cut — nhóm Ideal (cắt đẹp nhất) lại có trung vị giá thấp hơn Fair và Premium.

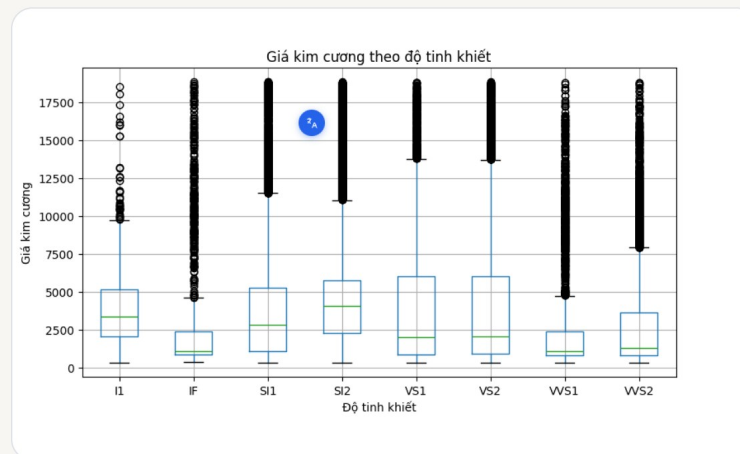
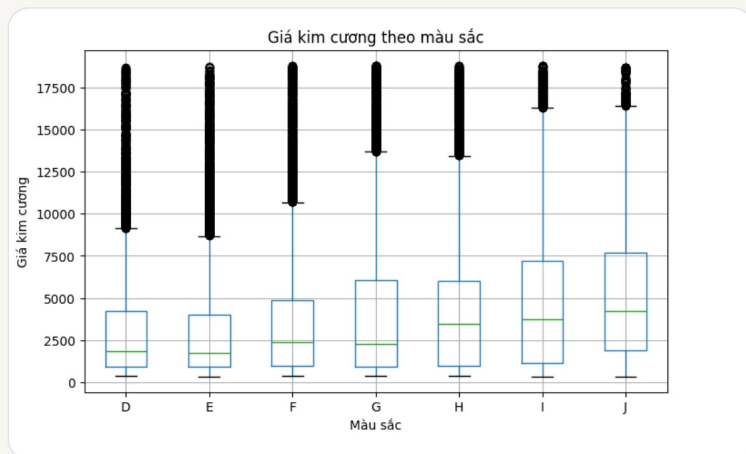
Ý nghĩa với bài toán?

Cut một mình không quyết định giá — hiện tượng này bắt nguồn từ tương quan ẩn với carat trong tập dữ liệu thực tế.

Quyết định xử lý

Không dùng Ordinal Encoding (giả định thứ tự tuyến tính); dùng One-Hot Encoding để mô hình tự học tương tác giữa cut và các biến khác.

Giá kim cương theo màu sắc & độ tinh khiết



Hình cho thấy gì? Cả hai boxplot (màu D→J, độ tinh khiết IF→I1) đều cho xu hướng ngược trực giác: nhóm "kém" hơn (I, J, SI1, SI2) lại có trung vị giá cao hơn nhóm "tốt nhất" (D, IF).

Ý nghĩa: Đây là hệ quả của tương quan với carat trong dữ liệu thị trường thực (kim cương carat lớn thường chấp nhận màu/độ tinh khiết thấp hơn) — không phải color/clarity không quan trọng.

Quyết định xử lý: Giữ cả 3 biến hạng mục (cut, color, clarity) dạng One-Hot để mô hình phi tuyến tự học tương tác đa biến với carat.

Làm sạch, mã hoá và chống rò rỉ dữ liệu

Kiểm tra & làm sạch

- 53.940 dòng, không có giá trị thiếu (isnull = 0)
- Phát hiện bản ghi lỗi vật lý: x, y hoặc z = 0 → loại bỏ
- Bỏ cột định danh "Unnamed: 0"
- Chia train/test 80/20, random_state = 42 (tái lập được)

Mã hoá & Pipeline chống rò rỉ

- 6 biến số (carat, depth, table, x, y, z): giữ nguyên (passthrough)
- 3 biến hạng mục (cut, color, clarity):
OneHotEncoder(handle_unknown="ignore")
- Encoder + mô hình gói chung trong 1 sklearn Pipeline
- Pipeline chỉ fit() trên train, test chỉ transform() → không rò rỉ dữ liệu

4 MÔ HÌNH HỒI QUY

Nguyên lý & tham số tinh chỉnh (GridSearchCV, CV = 3)

Linear Regression

Giá định price là tổ hợp tuyến tính của các đặc trưng đã mã hoá.

Tham số dò: fit_intercept, positive

Decision Tree

Chia dữ liệu đệ quy theo ngưỡng đặc trưng, dự đoán bằng giá trị trung bình ở mỗi lá.

Tham số dò: max_depth {5,10,15,20}, min_samples_split {2,5,10}

Random Forest

Kết hợp (bagging) nhiều cây quyết định độc lập, lấy trung bình để giảm phương sai.

Tham số dò: n_estimators {100,200}, max_depth {10,20,None}, min_samples_split {2,5}

Gradient Boosting

Xây tuần tự các cây yếu, mỗi cây sửa lỗi (residual) của cây trước theo gradient.

Tham số dò: n_estimators {100,200}, learning_rate {0.05, 0.1}, max_depth {2,3}

ĐÁNH GIÁ MÔ HÌNH

R², MAE, RMSE trên tập kiểm tra (test set)

R² — tỉ lệ phương sai của giá được mô hình giải thích; càng gần 1 càng khớp dữ liệu.

MAE — sai lệch tuyệt đối trung bình (USD); dễ hiểu, ít bị outlier chi phối.

RMSE — căn bậc hai sai số bình phương trung bình; phạt nặng các dự đoán lệch lớn.

Mô hình	R ²	MAE (USD)	RMSE (USD)
★ Random Forest	0.98	263.34	519.21
Decision Tree	0.98	321.90	628.22
Gradient Boosting	0.97	361.59	651.99
Linear Regression	0.92	791.70	1,127.92

Nguồn: ai-models/models/metadata.json — kết quả sau khi tinh chỉnh bằng GridSearchCV.

Vì sao Random Forest được chọn?

★ Random Forest Regressor — mô hình mặc định của AI Service

MAE & RMSE thấp nhất trong 4 mô hình (sai số trung bình ~263 USD trên giá kim cương trung bình ~3.930 USD, dưới 7%). $R^2 = 0,98$ ngang Decision Tree nhưng ổn định hơn nhờ lấy trung bình (bagging) nhiều cây, ít phụ thuộc vào một lần chia dữ liệu ngẫu nhiên.

Linear Regression — kém phù hợp

RMSE gấp ~2,2 lần Random Forest: quan hệ carat-price phi tuyến (Hình 2) khiến giả định tuyến tính không đủ mô tả dữ liệu.

Decision Tree — kém ổn định hơn

R^2 ngang Random Forest nhưng RMSE cao hơn ~21%: một cây đơn dễ bám nhiễu của tập train, kém ổn định khi dữ liệu thay đổi.

Gradient Boosting — không vượt trội

Gần Random Forest nhưng nhạy với `learning_rate/n_estimators` hơn, huấn luyện lâu hơn mà không giảm được sai số trên tập test này.

Từ Colab đến sản phẩm: kiến trúc 3 container

Artifacts trong ai-models/models/

- 4 file *.joblib — pipeline hoàn chỉnh (encoder + model) cho từng thuật toán
- metadata.json — tên model, MAE / RMSE / R^2 , ngày huấn luyện
- schema.json — kiểu dữ liệu & giá trị hợp lệ của 9 đặc trưng đầu vào

3 container Docker

- AI Service (FastAPI) — nạp 4 model vào RAM, /predict + /predict/<model>
- Backend (Express) — validate schema, sinh request_id, lưu MongoDB Atlas
- Frontend — chọn 1 trong 4 model, xem giá dự đoán theo thời gian thực

Kết luận: hệ thống đạt $R^2 = 0,98$ với mô hình Random Forest, đóng gói khép kín từ notebook nghiên cứu đến API phục vụ thực tế.